

Objectifs

Savoir écrire une servlet simple, savoir gérer les formulaires, savoir effectuer des redirections

- [Documentation JDBC](#)
- [Documentation Servlet](#)

Préambule

1. Supprimez toute installation antérieure de Tomcat qui pourrait subsister.
2. Téléchargez et installez une version 11 CORE en `tar.gz` du serveur [Tomcat](#) et décompressez la dans votre répertoire personnel. Renommez-le répertoire principal en `tomcat` pour faciliter ensuite les adressages.
3. Un serveur PostgreSQL est disponible sur la machine `psqlserv`. Un compte est disponible avec votre login habituel sans les points, et un mot de passe `'moi'`. Une base `but3` est à votre disposition.

```
psql -h nomMachine -U login nomBase
```

Exercice 1 : Test de la configuration

Q1. Créez un répertoire `tp501` dans `tomcat/webapps`.

Q2. Créez une page HTML `index.html` avec `<h1>All is ok !</h1>` dedans. Testez.

Q3. Recopiez `index.html` en `index.jsp`. Testez.

On reviendra plus tard sur les JSP

Luttons contre les idées reçues :

Vous constatez que pour faire du JEE on n'a besoin de rien ! c'est SUPER SIMPLE.

Q4. Une servlet, comme tout autre objet Java, doit toujours être en zone protégée dans `WEB-INF/classes`. Créez ce répertoire, puis créez une servlet `Test.java` qui dans sa méthode `service` affiche `All is ok !`. Testez.

C'est bon, vous savez tout faire ;-)

Q5. Par défaut Tomcat ne recharge pas les objets après compilation. Pour qu'il le fasse, créez un répertoire `META-INF` dans `tp501` avec un fichier `context.xml` contenant la ligne : `<Context reloadable="true" />`

```

tp501
|-- META-INF
| |-- context.xml
|-- WEB-INF
|   |-- classes
|       |-- Test.java
|       |-- Test.class
|-- index.html
|-- index.jsp

```

Exercice 2 : Basique

Q1. Connectez vous à votre base de données et créez une table `etudiant (id,nom,prenom,groupe)` avec quelques lignes dedans.

```

(1, 'AUFFRAY', 'Malo', 'M')
(2, 'DANJOUX', 'Theo', 'N')
(3, 'DELORME', 'Pauline', 'O')
(4, 'FAVEEUW', 'Alexandre', 'O')
....

```

Q2. Ecrire une servlet `Select.java` qui permet d'afficher le contenu de la table `etudiant`. Pour cela vous avez besoin d'un driver de base de données : Téléchargez le [driver JDBC Postgres](#) et placez le (directement ou via un lien symbolique) dans `tomcat/lib`

Q3. Ecrire un formulaire HTML `insert.html` contenant un formulaire de saisie pour cette table. Il appellera une servlet `Insert.java`.

Q4. Ecrire la servlet `Insert.java` qui s'occupe uniquement d'insérer les données passées en paramètre.

Exercice 3 : Normal

Q1. Transformez la servlet `Insert.java` de manière à ce que sa partie `doGet` affiche le formulaire et sa partie `doPost` le traite.

Cette fois le HTML est généré dynamiquement.

Q2. Modifiez `Select` de manière à ce que le formulaire soit affiché en bas de la table, et que sa validation ramène directement à l'affichage.

Maintenant on ne voit plus qu'une page (la liste), mais 2 servlets sont nécessaires à son bon fonctionnement.

Q3. Mettre en place une CSS qui centre les titres et met en surbrillance la ligne de la table sur laquelle passe le curseur de la souris.

Q4. Réalisez une requête `curl` en GET sur la servlet `Select` pour afficher les données de la page (y compris les headers).

Q5. Réalisez une requête `curl` en POST sur la servlet `Insert` pour saisir une nouvelle ligne

Exercice 4 : Avancé : PGJavaAdmin

Le but de cet exercice est de réaliser une petite application de type `phpmyadmin` ou `phppgadmin` permettant d'interagir avec ses tables dans un navigateur web. Le principe est grosso-modo identique à l'exercice précédent, mais cela doit fonctionner avec n'importe quelle table passée en paramètre.

Q1. Ecrire une servlet `Select.java` qui permet d'afficher n'importe quelle table dont le nom est passé en paramètre. La servlet doit bien sûr utiliser les `MetaData` pour connaître le nombre et les noms de colonnes.

Ex : tp501/servlet-Select?table=personne

On a besoin ici d'une seule servlet.

Q2. Ecrire une servlet `Insert.java` qui présente un formulaire permettant d'insérer une ligne dans une table dont le nom est passé en paramètre. Le formulaire n'est pas "en dur" mais calculé dynamiquement en fonction du nom de la table à l'aide de la Métabase. On utilisera avantageusement les méthodes `doGet` pour afficher le formulaire et `doPost` pour effectuer l'insertion.

Ex : tp501/servlet-Insert?table=personne.

Pour simplifier et éviter les tests de types, envoyez tout en txt, en fabriquant la requête par concaténation et sans passer par les setters.

Deux méthodes ou deux servlets ; l'une fabrique le formulaire HTML, l'autre exécute la requête. Utilisation de champs hidden

Q3. Créez une servlet `Delete.java` qui permet de détruire une ligne d'une table. La table sera passée en paramètre et la ligne sera repérée par l'ensemble de ses valeurs.(dans un premier temps, on pourra considérer pour simplifier que la première colonne correspond à la clé et qu'on ne passe que cette clé ; ensuite on généralise à l'ensemble des valeurs).

Ex : tp501/servlet-Delete?table=personne&cle=12

Cette fois on génère un formulaire par ligne.

Q4. Ajoutez à la page `Select` un lien `Del` en fin de chaque ligne permettant d'appeler cette Servlet pour effacer la ligne correspondante.